



AI-Driven Portfolio Optimization Techniques for Constructing a Smart Index Fund to track S&P 100

Mohammad Shehnaj | Shayshank Rathore
x23305762 | x23348186
MSCAI1



Introduction



Traditional index funds track indices like the S&P 100 using all constituent stocks.

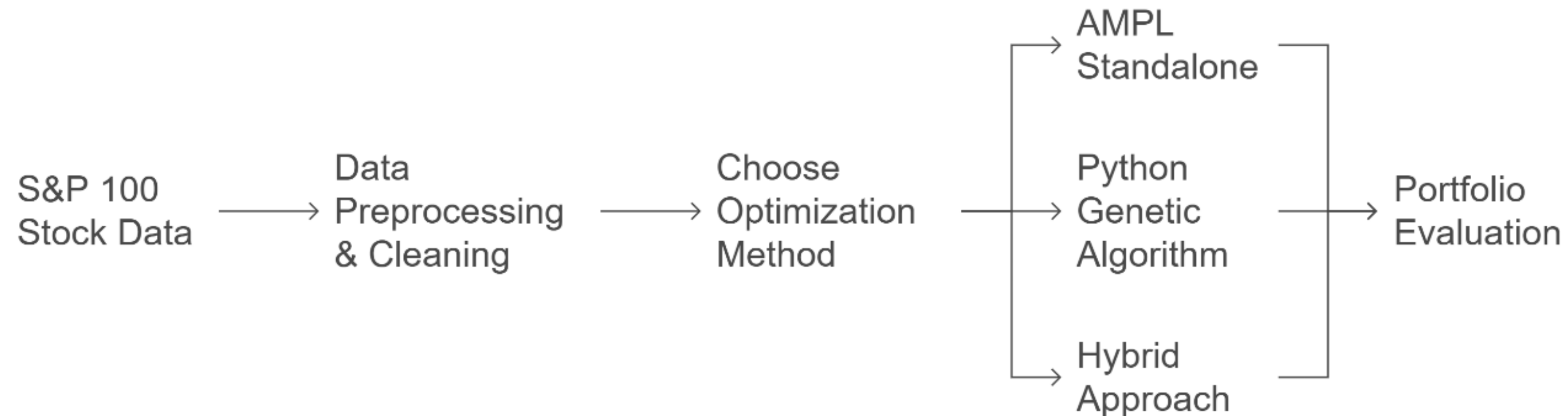


This project aims to create a smart index fund reducing number of assets ($q < 100$) stocks while preserving performance of ETF OEF.



Compare three optimization approaches - AMPL, Python GA, Hybrid to minimize tracking error and evaluate performance.

Portfolio Optimization Process



Problem Statement

- **Goal:** Mimic the performance of the S&P 100 using only $q < 100$ stocks.
- **Objective:** Minimize tracking error between the constructed portfolio and benchmark.

$$\min_{\mathbf{w}, \mathbf{x}} \sqrt{\frac{1}{T} \sum_{t=1}^T \left(\sum_{i \in S} x_i \cdot w_i \cdot r_i(t) - R_b(t) \right)^2}$$

- T : number of trading days
- S : full set of S&P 100 stocks
- $S_q \subset S$: subset of selected stocks, where $|S_q| = q$
- $r_i(t)$: return of stock $i \in S_q$ on day t
- $R_b(t)$: benchmark ETF return on day t
- w_i : weight of stock $i \in S_q$
- $R_p(t) = \sum_{i \in S_q} w_i \cdot r_i(t)$: portfolio return on day t

Methodology



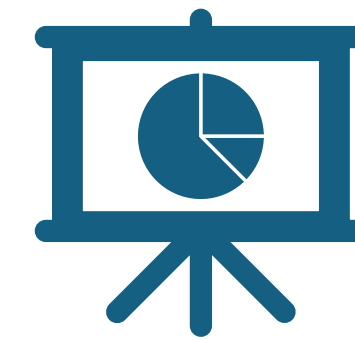
Step 1: Data
Collection (Yahoo
Finance API)



Step 2: Preprocessing
(returns, alignment,
cleaning)



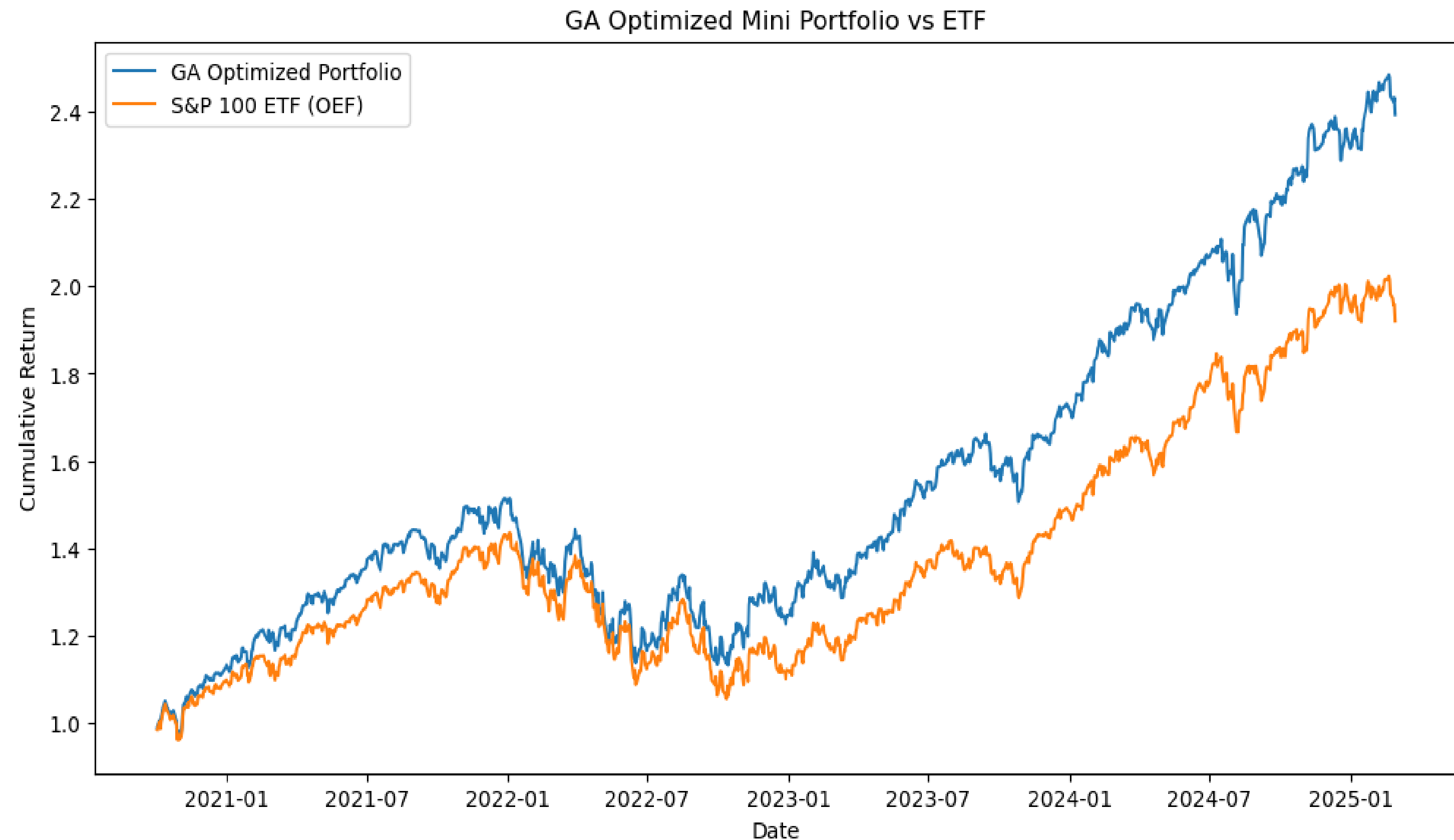
Step 3: Optimization
using 3 techniques



Step 4: Evaluation
using metrics and
visualizations

Approach 1: Python-Based Genetic Algorithm

- *Tools:* Python, DEAP, NumPy, Pandas
- *Strategy:* 20 selected stock indices + weights
- *Output:*
 - ✓ Tracking Error: 0.0034
 - ✓ Correlation: 0.9487
 - ✓ Final Cumulative Return: 1.9144



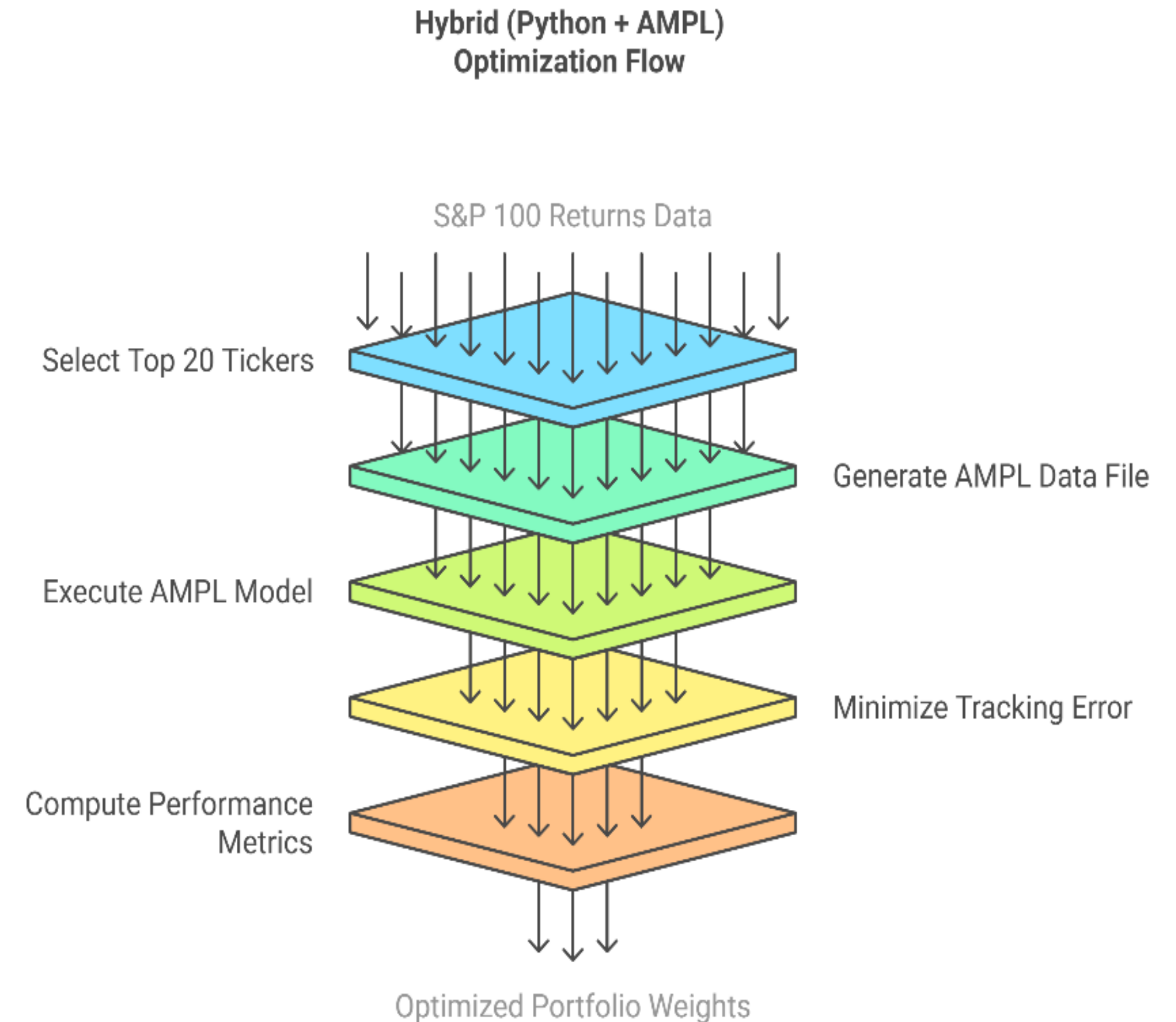
Approach 2: AMPL Standalone Optimization

- AMPL model with CPLEX solver
- *Input:* .mod and .dat files with constraints and returns.
- *Output:*
 - Tracking Error: 0.0026
 - Correlation: 0.9718
 - Final Cumulative Return: 2.6031

```
PS C:\Users\moham\AMPL\ampl> ampl
ampl: reset;
ampl: option solver cplex;
ampl: include run_index.run;
CPLEX 22.1.1: optimal solution; objective 0.007438187292
0 simplex iterations
w [*] :=
AAPL 0.148735
ABT 0.0325095
AVGO 0.0610043
AXP 0.0354142
BKNG 0.0139669
BRK-B 0.109884
C 0.0569377
CL 0.0595313
GOOG 0.0697454
GOOGL 0.0445109
INTU 0.022565
LIN 0.0260445
MCD 0.0629184
MO 0.0407181
MRK 0.0471924
NFLX 0.0256679
NOW 0.033732
NVDA 0.0602021
TMUS 0.0299315
UNP 0.0187891
;
```


Approach 3: Hybrid Python + AMPL

- Python generates .dat file, calls AMPL via subprocess.
- AMPL performs optimization & results read back by Python.
- Combines data handling and solving power.
- *Output:*
 - Tracking Error: 0.0026
 - Correlation: 0.9718
 - Final Cumulative Return: 2.6031



Performance Comparison



All three methods use the same dataset and objective function



Python GA:
Tracking Error = 0.0034
Correlation = 94.87%
Return = 1.91x



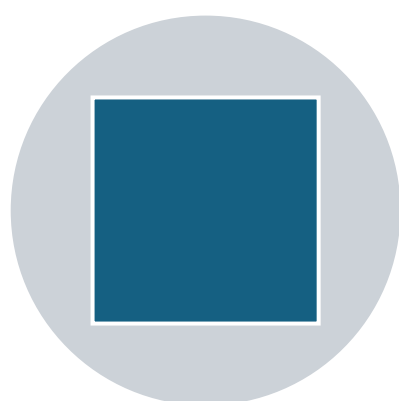
AMPL:
Tracking Error = 0.0026
Correlation = 97.18%
Return = 2.60x



Hybrid:
Tracking Error = 0.0026
Correlation = 97.18%
Return = 2.60x



q=20 found optimal

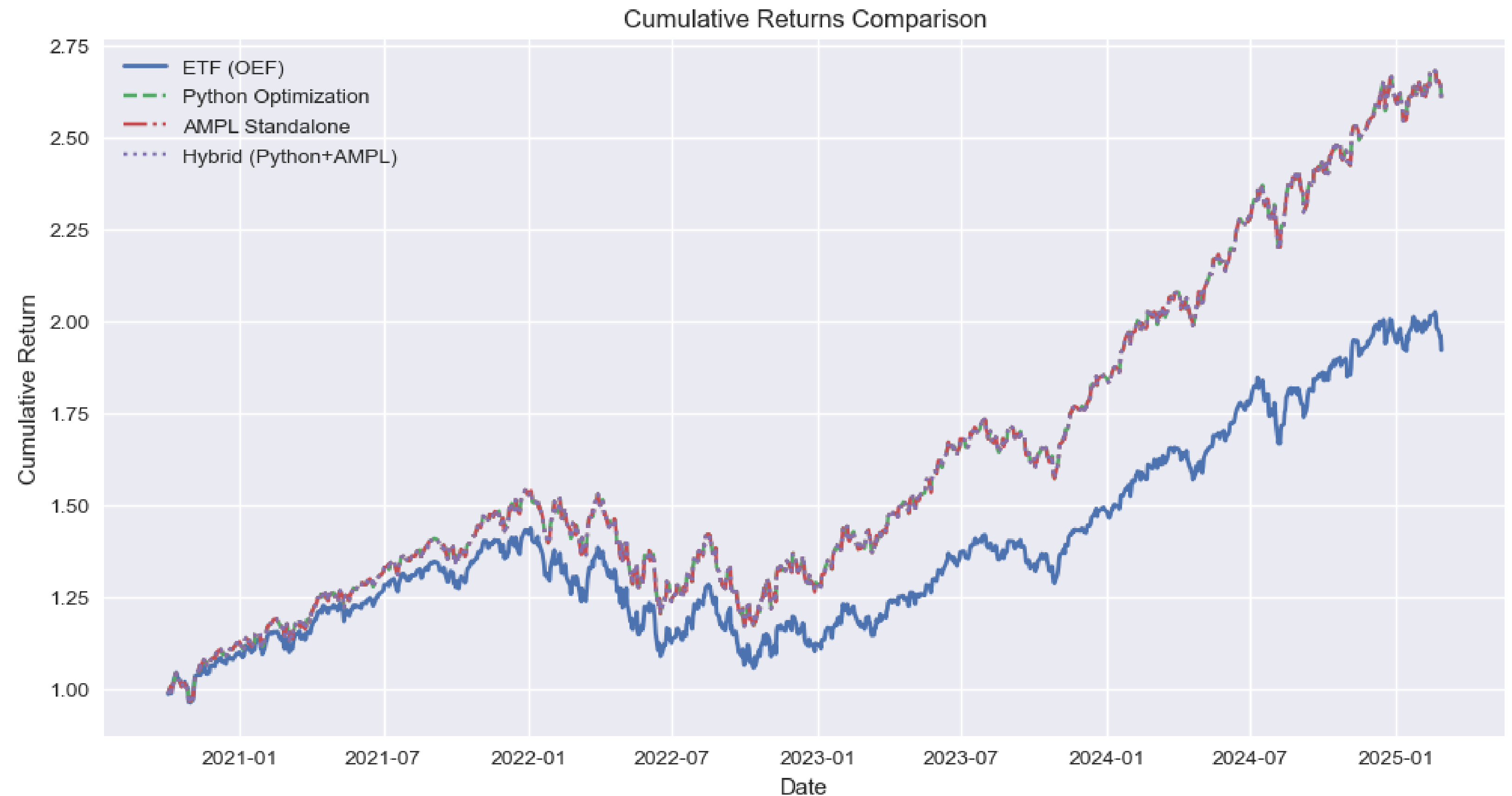


All 3 approaches perform equally well.

	Tracking Error	Correlation	Final Cumulative Return
Python Optimization	0.0034	0.9487	1.9144
AMPL Standalone	0.0026	0.9718	2.6031
Hybrid (Python+AMPL)	0.0026	0.9718	2.6031

Insights & Observations

- Increasing q reduces TE but 20 is a practical trade-off.
- All methods show similar performance due to shared constraints.
- GA allows flexibility, AMPL is precise, Hybrid combines both.



Conclusion & Future Work

₹ Optimized portfolio of 20 stocks closely tracks S&P 100.

🤖 *Python GA*: Flexible, exploratory.

\$ *AMPL*: Robust and fast.

👛 *Hybrid*: Best of both worlds.

👤 *Future*: Sharpe Ratio, Sector Constraints, Reinforcement Learning.